

文献阅读报告：Decision Transformer

——通过序列建模实现离线强化学习

王志豪 周开轩 张沁怡

2026-05-31

目录

1	文献基本信息	1
2	研究背景、核心问题与贡献	1
2.1	离线强化学习的形式化描述	1
2.2	核心问题与方法创新	2
3	关键算法描述	2
3.1	轨迹表示 (Trajectory Representation)	2
3.2	架构设计	3
3.3	数学形式化	3
3.3.1	输入嵌入	3
3.3.2	自注意力机制	3
3.3.3	训练目标	3
3.4	算法伪代码	4
3.5	与 TD-learning 的本质区别	4
4	原论文实验设置与主要结果	5
4.1	实验一：Atari 离散控制	5
4.1.1	实验设置	5
4.1.2	基准归一化分数	5
4.1.3	原始分数	5
4.1.4	Atari 超参数配置	6
4.2	实验二：OpenAI Gym / D4RL 连续控制	6
4.2.1	数据集设置	6
4.2.2	评估指标	6
4.2.3	完整实验结果	6
4.2.4	OpenAI Gym 超参数配置	7
4.3	实验三：Key-to-Door 长程信用分配	8
4.4	分析实验与消融研究	8
4.4.1	上下文长度消融	8
4.4.2	Percentile Behavior Cloning (%BC) 对比	8
4.4.3	稀疏/延迟奖励鲁棒性	9
4.4.4	回报分布建模能力	9
4.4.5	Transformer 作为 Critics 的能力	9
5	小组思考与评价	10
5.1	方法论评价	10
5.2	优势	10

5.3 局限与讨论	10
5.4 本课程复现方案	10
6 参考文献	10

1 文献基本信息

项目	内容
论文	Decision Transformer: Reinforcement Learning via Sequence Modeling
作者	Lili Chen*, Kevin Lu*, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Michael Laskin, Pieter Abbeel, Aravind Srinivas†, Igor Mordatch†
机构	UC Berkeley / Facebook AI Research / Google Brain
年份	2021 (NeurIPS 2021)
方向	Offline Reinforcement Learning; Transformer; Sequence Modeling
论文链接	https://arxiv.org/abs/2106.01345
官方代码	https://github.com/kzl/decision-transformer

* equal contribution, † equal advising

2 研究背景、核心问题与贡献

2.1 离线强化学习的形式化描述

考虑马尔可夫决策过程 (MDP), 定义为元组 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R})$, 其中:

- $\mathcal{S}$: 状态空间, $s \in \mathcal{S}$
- $\mathcal{A}$: 动作空间, $a \in \mathcal{A}$
- $\mathcal{P}(s' | s, a)$: 状态转移概率
- $\mathcal{R}(s, a)$: 奖励函数, $r_t = \mathcal{R}(s_t, a_t)$

一条轨迹由状态、动作和奖励序列组成:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T)$$

轨迹在时刻 t 的回报 (return) 定义为从该时刻到终止的累积奖励:

$$R_t = \sum_{t'=t}^T r_{t'}$$

强化学习的目标是学习一个策略 $\pi(a | s)$, 最大化期望回报:

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^T r_t \right]$$

在**离线强化学习** (Offline RL) 设定下, 智能体无法与环境交互探索, 只能访问一个固定的、由任意策略收集的轨迹数据集 $\mathcal{D} = \{\tau_i\}_{i=1}^N$ 。这使得传统基于 TD-learning 的方法面临分布偏移 (distribution shift) 和价值过估计 (value overestimation) 的严重挑战。

2.2 核心问题与方法创新

传统 RL 方法的核心在于通过 Bellman 方程迭代优化：

$$Q(s, a) \leftarrow r + \gamma \max_{a'} Q(s', a')$$

Decision Transformer 提出的范式转换是：将 RL 重新定义为条件序列建模问题，从而绕过 Bellman 备份和动态规划。

论文的三大贡献：

1. **范式抽象**：将 RL 轨迹表示为 return-to-go、state、action 交替排列的序列，用 causal Transformer 做自回归条件生成
2. **条件生成**：以 desired return $\widehat{R}$ 作为条件，无需显式价值函数即可生成达到目标回报的动作序列
3. **经验验证**：在 Atari (离散控制)、OpenAI Gym/D4RL (连续控制) 和 Key-to-Door (长程信用分配) 三大类离线 RL 基准上，匹配或超越 state-of-the-art 的 model-free offline RL 方法，且架构简洁

3 关键算法描述

3.1 轨迹表示 (Trajectory Representation)

时序差分学习 (TD-learning) 需要折扣因子 γ 来避免无限视野下发散，但这可能引入短视行为。Decision Transformer 通过无折扣的 return-to-go 解决此问题：

$$\widehat{R}_t = \sum_{t'=t}^T r_{t'} \quad (\text{no discounting})$$

轨迹被重组为三模态交织序列：

$$\tau_{\text{DT}} = (\widehat{R}_1, s_1, a_1, \widehat{R}_2, s_2, a_2, \dots, \widehat{R}_T, s_T, a_T)$$

其中 $\widehat{R}_t$ 作为条件信息，在测试时可设定目标值以引导期望的行为水平。

3.2 架构设计

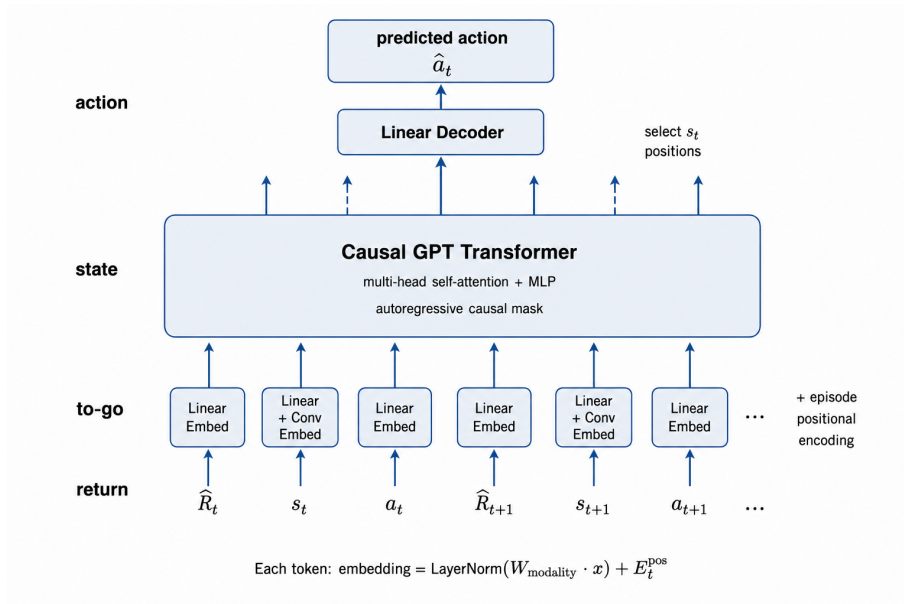


Figure 1: Decision Transformer 架构图。States, actions 和 returns 分别通过模态特定的线性嵌入层处理后，加上学到的 episode 时间步位置编码。3K 个 token (返回、状态、动作各 K 个) 被送入 GPT 架构，通过 causal self-attention mask 自回归地预测动作。仅 state token 位置的隐状态被提取用于动作预测。

3.3 数学形式化

3.3.1 输入嵌入

给定上下文长度 K ，输入序列包含 $3K$ 个 token。对每个模态学习独立的线性嵌入层：

$$e_t^R = W_R \widehat{R}_t + b_R, \quad e_t^s = W_s s_t + b_s, \quad e_t^a = W_a a_t + b_a$$

所有嵌入维度为 d_{model} 。对视觉输入 (Atari)，状态经过 DQN 卷积编码器：

$$e_t^s = \text{ConvEncoder}_{\text{DQN}}(s_t)W_s + b_s$$

随后叠加上学到的按时间步 (per-timestep) 的位置编码 (同一时间步的三个 token 共享相同的位置编码)：

$$\tilde{e}_t^R = e_t^R + E_t^{\text{pos}}, \quad \tilde{e}_t^s = e_t^s + E_t^{\text{pos}}, \quad \tilde{e}_t^a = e_t^a + E_t^{\text{pos}}$$

3.3.2 自注意力机制

Causal GPT Transformer 包含 L 层自注意力，每层输出为：

$$z_i^{(\ell)} = \sum_{j=1}^i \text{softmax} \left(\frac{\langle q_i, k_j \rangle}{\sqrt{d_k}} \right)_j \cdot v_j$$

其中 $q_i = \tilde{e}_i W_Q, k_i = \tilde{e}_i W_K, v_i = \tilde{e}_i W_V$ 。Causal mask 保证 $j \leq i$ ，保留了轨迹的时序因果性。

这种注意力机制自然地实现了信用分配 (credit assignment)：query-key 之间的相似性隐式建立了状态与回报之间的关联。

3.3.3 训练目标

Decision Transformer 的训练目标是条件动作预测。取 state token 对应位置的隐状态作为动作预测头的输入：

$$\hat{a}_t = \text{LinearDecoder}(h_t^s), \quad h_t^s = \text{Transformer}(\{\tilde{e}_{t'}\}_{t-K+1}^t)[s_t]$$

- 连续动作：最小化均方误差 (MSE)

$$\mathcal{L}_{\text{cont}} = \frac{1}{K} \sum_{t=1}^K \|\hat{a}_t - a_t\|_2^2$$

- 离散动作：最小化交叉熵 (Cross-Entropy)

$$\mathcal{L}_{\text{disc}} = -\frac{1}{K} \sum_{t=1}^K \log p_{\theta}(a_t \mid \widehat{R}_{1:t}, s_{1:t}, a_{1:t-1})$$

训练采用小批量随机梯度下降，优化目标为：

$$\theta^* = \arg \min_{\theta} \mathbb{E}_{\tau \sim \mathcal{D}} [\mathcal{L}(\theta; \tau)]$$

注意：DT 不预测状态和回报，论文发现预测这些 token 并不能提升性能。

3.4 算法伪代码

```

1: Input: 离线数据集  $\mathcal{D}$ , 上下文长度  $K$ , 目标回报  $\widehat{R}_{\text{target}}$ 
2: Parameters: Transformer  $\theta$ , 嵌入层  $\{W_R, W_s, W_a, E^{\text{pos}}\}$ , 解码器  $W_{\text{pred}}$ 
3: function DECISIONTRANSFORMER( $\widehat{\mathbf{R}}, \mathbf{s}, \mathbf{a}, \mathbf{t}$ ) ▷  $\widehat{\mathbf{R}}, \mathbf{s}, \mathbf{a} \in \mathbb{R}^{K \times d}, \mathbf{t} \in \mathbb{N}^K$ 
4:    $\mathbf{e}^{\text{pos}} \leftarrow \text{embed\_t}(\mathbf{t})$  ▷ per-timestep position encoding
5:    $\mathbf{e}^R \leftarrow \text{LayerNorm}(W_R \widehat{\mathbf{R}}) + \mathbf{e}^{\text{pos}}$ 
6:    $\mathbf{e}^s \leftarrow \text{LayerNorm}(W_s \mathbf{s}) + \mathbf{e}^{\text{pos}}$ 
7:    $\mathbf{e}^a \leftarrow \text{LayerNorm}(W_a \mathbf{a}) + \mathbf{e}^{\text{pos}}$ 
8:   ▷ 交织为  $(\widehat{R}_1, s_1, a_1, \dots, \widehat{R}_K, s_K, a_K)$ 
9:    $\mathbf{E} \leftarrow \text{interleave}(\mathbf{e}^R, \mathbf{e}^s, \mathbf{e}^a)$  ▷ shape:  $(3K, d_{\text{model}})$ 
10:   $\mathbf{H} \leftarrow \text{CausalGPT}(\mathbf{E})$  ▷ multi-head self-attention + MLP
11:   $\mathbf{H}_s \leftarrow \text{extract\_state\_tokens}(\mathbf{H})$  ▷ 取  $s_t$  位置的隐状态
12:  return  $W_{\text{pred}} \mathbf{H}_s$  ▷ 预测动作  $\hat{\mathbf{a}}$ 
13: end function

// Training Loop
14: for  $(\widehat{\mathbf{R}}, \mathbf{s}, \mathbf{a}, \mathbf{t}) \in \text{DataLoader}(\mathcal{D}, \text{batch\_size}, K)$  do
15:    $\hat{\mathbf{a}} \leftarrow \text{DecisionTransformer}(\widehat{\mathbf{R}}, \mathbf{s}, \mathbf{a}, \mathbf{t})$ 
16:    $\mathcal{L} \leftarrow \frac{1}{K} \sum \|\hat{\mathbf{a}} - \mathbf{a}\|_2^2$  ▷ MSE for continuous actions
17:   optimizer.zero_grad();  $\mathcal{L}$ .backward(); optimizer.step()
18: end for

// Evaluation Loop (Autoregressive Generation)
19:  $\widehat{R}, s, a, t, \text{done} \leftarrow [\widehat{R}_{\text{target}}], [\text{env.reset()}], [], [1], \text{False}$ 
20: while  $\neg \text{done}$  do
21:    $\text{action} \leftarrow \text{DecisionTransformer}(\widehat{R}[-K:], s[-K:], a[-K:], t[-K:])[ -1]$ 
22:    $s_{\text{new}}, r, \text{done} \leftarrow \text{env.step}(\text{action})$ 
23:    $\widehat{R}.\text{append}(\widehat{R}[-1] - r)$  ▷ 递减 return-to-go
24:    $s.\text{append}(s_{\text{new}}), a.\text{append}(\text{action}), t.\text{append}(\text{len}(\widehat{R}))$ 
25: end while

```

Algorithm 1: Decision Transformer: 训练与评估 (连续动作)

3.5 与 TD-learning 的本质区别

Decision Transformer 与基于 Bellman 备份的传统 RL 方法有根本性的不同:

TD-learning (CQL 等)	Decision Transformer
优化 Bellman 残差: $\mathcal{L}_{\text{TD}} = \mathbb{E}[(Q - \mathcal{B}^*Q)^2]$	优化动作预测: $\mathcal{L}_{\text{DT}} = \mathbb{E}\ \hat{a} - a\ ^2$
需要价值悲观/行为正则化	无需额外的保守性约束
通过 Bellman 备份传播信用	通过 self-attention 直接建立状态-回报关联
使用折扣因子 γ 保证收敛	无折扣, 直接使用 return-to-go
易受 deadly triad 影响	绕过 bootstrap, 避免 deadly triad

4 原论文实验设置与主要结果

4.1 实验一：Atari 离散控制

4.1.1 实验设置

- 数据集: DQN-replay 数据集的 1% 子样本 (约 5×10^5 条 transition, 对应在线 DQN 训练中 5×10^7 条 transition 的 1%)
- 评估指标: $\text{Gamer-normalized score} = \frac{\text{agent_score} - \text{random_score}}{\text{professional_gamer_score} - \text{random_score}} \times 100$, 即 100 代表职业玩家水平, 0 代表随机策略
- 基准方法: CQL, REM, QR-DQN, Behavior Cloning (BC)
- 超参数: 上下文长度 $K = 30$ (Pong 为 $K = 50$), 3 个随机种子报告均值 $\pm$ 标准差

4.1.2 基准归一化分数

Table 2: Gamer-normalized scores for the 1% DQN-replay Atari dataset. 均值 $\pm$ 标准差 (3 seeds). 最佳均值加粗标注.

Game	DT (Ours)	CQL	QR-DQN	REM	BC
Breakout	267.5 $\pm$ 97.5	211.1	17.1	8.9	138.9 $\pm$ 61.7
Qbert	15.4 $\pm$ 11.4	104.2	0.0	0.0	17.3 $\pm$ 14.7
Pong	106.1 $\pm$ 8.1	111.9	18.0	0.5	85.2 $\pm$ 20.0
Seaquest	2.5 $\pm$ 0.4	1.7	0.4	0.7	2.1 $\pm$ 0.3

4.1.3 原始分数

Table 3: Atari 1% DQN-replay 原始分数 (Raw scores). 均值 $\pm$ 标准差 (3 seeds).

Game	DT (Ours)	CQL	QR-DQN	REM	BC
Breakout	76.9 $\pm$ 27.3	61.1	6.8	4.5	40.9 $\pm$ 17.3
Qbert	2215.8 $\pm$ 1523.7	14012.0	156.0	160.1	2464.1 $\pm$ 1948.2
Pong	17.1 $\pm$ 2.9	19.3	-14.5	-20.8	9.7 $\pm$ 7.2
Seaquest	1129.3 $\pm$ 189.0	779.4	250.1	370.5	968.6 $\pm$ 133.8

结论: Decision Transformer 在 4 个游戏中的 3 个上与 CQL 持平, 在全部 4 个游戏上优于或匹配 REM、QR-DQN 和 BC。

4.1.4 Atari 超参数配置

Table 4: Decision Transformer Atari 实验超参数

超参数	取值
Transformer 层数 L	6
注意力头数 h	8
嵌入维度 d_{model}	128
Batch size	128 (Breakout, Qbert, Seaquest), 512 (Pong)
上下文长度 K	30 (Breakout, Qbert, Seaquest), 50 (Pong)
目标回报 $\widehat{R}_{\text{target}}$	90 (Breakout, $\approx 1 \times \max$), 2500 (Qbert, $\approx 5 \times \max$), 20 (Pong, $\approx 1 \times \max$), 1450 (Seaquest, $\approx 5 \times \max$)
非线性激活	ReLU (编码器), GeLU (其他)
编码器架构	3 层 Conv: channels [32, 64, 64], filters [$8 \times 8, 4 \times 4, 3 \times 3$], strides [4, 2, 1]
Dropout	0.1
学习率	6×10^{-4}
Adam 参数	$\beta_{1,2} = (0.9, 0.95)$
梯度裁剪	1.0
权重衰减	0.1
学习率调度	Linear warmup + cosine decay
最大训练 epochs	5
训练总 token 数	$2 \times 500000 \times K$

4.2 实验二：OpenAI Gym / D4RL 连续控制

4.2.1 数据集设置

D4RL 基准提供三种离线数据分布：

1. **Medium**: 由性能约为专家策略 $\frac{1}{3}$ 的”中等”策略生成的 10^6 时间步
2. **Medium-Replay**: 训练至中等策略水平期间的 replay buffer (约 2.5×10^4 – 4×10^5 时间步)
3. **Medium-Expert**: 中等策略生成的 10^6 + 专家策略生成的 10^6 时间步 (共 2×10^6)

4.2.2 评估指标

标准化分数 = $\frac{\text{score} - \text{random_score}}{\text{expert_score} - \text{random_score}} \times 100$, 即 100 为专家水平。

4.2.3 完整实验结果

结论：在 OpenAI Gym/D4RL 连续控制实验中，Decision Transformer 在多数环境-数据设置中取得最高或接近最高的归一化分数，并在总体平均分上达到 69.2，高于 CQL 的 54.2 和 BC 的 47.7。需要注意，Reacher 并非 D4RL 标准任务，而是论文额外构造的 2D sparse-reward 环境。

Table 5: D4RL 连续控制任务完整结果。均值 $\pm$ 标准差 (3 seeds)。DT 在大部分任务上达到最高分数。

Dataset	Environment	DT (Ours)	CQL	BEAR	BRAC-v	AWR	BC
Med-Expert	HalfCheetah	86.8 $\pm$ 1.3	62.4	53.4	41.9	52.7	59.9
	Hopper	107.6 $\pm$ 1.8	111.0	96.3	0.8	27.1	79.6
	Walker	108.1 $\pm$ 0.2	98.7	40.1	81.6	53.8	36.6
	Reacher	89.1 $\pm$ 1.3	30.6	–	–	–	73.3
Medium	HalfCheetah	42.6 $\pm$ 0.1	44.4	41.7	46.3	37.4	43.1
	Hopper	67.6 $\pm$ 1.0	58.0	52.1	31.1	35.9	63.9
	Walker	74.0 $\pm$ 1.4	79.2	59.1	81.1	17.4	77.3
	Reacher	51.2 $\pm$ 3.4	26.0	–	–	–	48.9
Med-Replay	HalfCheetah	36.6 $\pm$ 0.8	46.2	38.6	47.7	40.3	4.3
	Hopper	82.7 $\pm$ 7.0	48.6	33.7	0.6	28.4	27.6
	Walker	66.6 $\pm$ 3.0	26.7	19.2	0.9	15.5	36.9
	Reacher	18.0 $\pm$ 2.4	19.0	–	–	–	5.4
Avg. (w/o Reacher)		74.7	63.9	48.2	36.9	34.3	46.4
Avg. (All Settings)		69.2	54.2	–	–	–	47.7

4.2.4 OpenAI Gym 超参数配置

Table 6: Decision Transformer OpenAI Gym 实验超参数

超参数	取值
Transformer 层数 L	3
注意力头数 h	1
嵌入维度 d_{model}	128
非线性激活	ReLU
Batch size	64
上下文长度 K	20 (HalfCheetah, Hopper, Walker), 5 (Reacher)
目标回报 $\widehat{R}_{\text{target}}$	6000 (HalfCheetah, 50%), 3600 (Hopper), 5000 (Walker), 50 (Reacher)
Dropout	0.1
学习率	10^{-4}
梯度裁剪	0.25
权重衰减	10^{-4}
学习率调度	Linear warmup (前 10^5 steps)
训练步数	10^5 gradient steps
优化器	AdamW (PyTorch defaults)

Table 7: Behavior Cloning 超参数 (OpenAI Gym). MLP 架构用于 BC 和 %BC 实验.

超参数	取值
网络层数	3
嵌入维度	256
非线性激活	ReLU
Batch size	64
训练步数	2.5×10^4

4.3 实验三：Key-to-Door 长程信用分配

Key-to-Door 是一个网格环境，分为三个阶段：(1) 智能体被放置在有一把钥匙的房间；(2) 空房间；(3) 有一扇门的房间。智能体仅在第三阶段到达门时获得二元奖励，但前提是在第一阶段捡起了钥匙。

此任务的挑战在于信用必须从 episode 结束传播回到开始，中间隔了大量的无关动作。

Table 8: Key-to-Door 环境成功率。使用 hindsight return 信息的方法 (DT, %BC) 可以学到成功策略，而 TD learning 难以完成信用分配。

Dataset	DT (Ours)	CQL	BC	%BC	Random
1K Random Trajectories	71.8%	13.1%	1.4%	69.9%	3.1%
10K Random Trajectories	94.6%	13.3%	1.6%	95.1%	3.1%

结论：Decision Transformer 仅基于随机游走数据即可学到接近最优的路径 (94.6% 成功率 @10K trajectories)，而 CQL 仅 13.3%，证明了 self-attention 在长程信用分配上的优势。

4.4 分析实验与消融研究

4.4.1 上下文长度消融

Table 9: 上下文长度消融实验。 $K = 1$ 时性能显著下降，验证了长上下文建模的重要性。

Game	DT ($K = 30/50$)	DT ($K = 1$, no context)
Breakout	267.5 ± 97.5	73.9 ± 10.0
Qbert	15.1 ± 11.4	13.6 ± 11.3
Pong	106.1 ± 8.1	2.5 ± 0.2
Seaquest	2.5 ± 0.4	0.6 ± 0.1

分析： $K = 1$ 退化为类似 Kumar et al. Reward-Conditioned Policies 的方法。长上下文使 Transformer 能够识别当前轨迹对应的策略分布模式 (policy-mode identification)，从而提升学习和训练动态。

4.4.2 Percentile Behavior Cloning (%BC) 对比

为探究 DT 是否只是对数据子集做行为克隆，论文提出 %BC 方法 (仅在 episode 回报前 $X\%$ 的时间步上做 BC)。

关键发现：

1. 高数据量场景 (D4RL)：最优 %BC 可与 DT 竞争，表明 DT 在数据充足时能自动关注高回报数据子集
2. 低数据量场景 (Atari, 1% DQN-replay)：%BC 远弱于 DT，表明 DT 通过利用全部轨迹提升泛化能力，即使这些轨迹与目标回报条件不相似
3. 选择最优 %BC 子集需要环境交互 (试错)，而 DT 通过 return conditioning 自动实现此能力

Table 10: %BC vs DT on D4RL Medium datasets. DT 可与最优%BC 竞争，但无需选择最优子集的先验知识。

Dataset	Env	DT	10%BC	25%BC	40%BC	100%BC
Medium	HalfCheetah	42.6	42.9	43.0	43.1	43.1
	Hopper	67.6	65.9	65.2	65.3	63.9
	Walker	74.0	78.8	80.9	78.8	77.3
	Reacher	51.2	51.0	48.9	58.2	58.4
Med-Replay	HalfCheetah	36.6	40.8	40.9	41.1	4.3
	Hopper	82.7	70.6	58.6	31.0	27.6
	Walker	66.6	70.4	70.9	67.2	36.9
	Reacher	18.0	33.1	16.2	10.7	5.4

Table 11: %BC vs DT on Atari. 在低数据量设定下，DT 显著优于所有%BC 变体。

Game	DT	10%BC	25%BC	40%BC	100%BC
Breakout	267.5 ± 97.5	28.5 ± 8.2	73.5 ± 6.4	108.2 ± 67.5	138.9 ± 61.7
Qbert	15.4 ± 11.4	6.6 ± 1.7	16.0 ± 13.8	11.8 ± 5.8	17.3 ± 14.7
Pong	106.1 ± 8.1	2.5 ± 0.2	13.3 ± 2.7	72.7 ± 13.3	85.2 ± 20.0
Seaquest	2.5 ± 0.4	1.1 ± 0.2	1.1 ± 0.2	1.6 ± 0.4	2.1 ± 0.3

4.4.3 稀疏/延迟奖励鲁棒性

将 D4RL Hopper 的稠密奖励替换为仅在 episode 结束时给出累积回报的稀疏奖励。

Table 12: D4RL Hopper 延迟奖励实验结果。DT 和 imitation learning 方法几乎不受影响，而 CQL 在延迟奖励下崩溃。

Dataset	Env	Delayed (Sparse)		Original (Dense)	
		DT	CQL	DT	CQL
Medium-Expert	Hopper	107.3 ± 3.5	9.0	107.6	111.0
Medium	Hopper	60.7 ± 4.5	5.2	67.6	58.0
Medium-Replay	Hopper	78.5 ± 3.7	2.0	82.7	48.6

分析：TD-learning 依赖稠密奖励信号来传播价值。DT 不依赖逐步奖励进行训练 (仅需 return-to-go 作为条件)，因此对奖励稀疏性天然鲁棒。BC 方法因为完全忽略奖励也表现良好，但无法像 DT 一样通过目标回报控制行为水平。

4.4.4 回报分布建模能力

论文测试了 DT 对 return-to-go token 的理解能力，通过在宽范围内变化目标回报并测量实际累积回报。结果显示：

- 在 Pong, HalfCheetah, Walker 等任务上，DT 生成的实际回报几乎完美匹配目标回报
- 在 Seaquest 上，DT 可以设置高于数据集中最大 episode 回报的目标，并能通过外推生成超出训练数据水平的轨迹 (extrapolation)
- 这表明 DT 对 return-to-go 条件具有较强的分布建模能力，并在部分任务上表现出一定外推能力

4.4.5 Transformer 作为 Critics 的能力

修改 DT 使其同时输出 return token 和 action token，并在 Key-to-Door 上分析注意力模式：

- Transformer 在 episode 过程中持续更新奖励预测概率
- 自注意力权重在关键事件 (捡钥匙、到达门) 附近出现峰值, 表明模型隐式形成了状态-回报关联
- 这解释了为什么 DT 不需要显式 Bellman 备份就能完成信用分配

5 小组思考与评价

5.1 方法论评价

Decision Transformer 的创新性在于改变了强化学习问题的建模语言。从数学角度看:

- **传统范式:** $\pi^* = \arg \max_{\pi} \mathbb{E} [\sum \gamma^t r_t]$, 通过迭代 Bellman 算子 $\mathcal{B}^*$ 求解
- **DT 范式:** $p(a_t | \bar{R}, s_{1:t}, a_{1:t-1})$, 通过自回归序列建模求解

这一范式转换使 RL 与 Transformer、大语言模型的预训练-微调范式直接对齐, 为后续的 Robot Foundation Model、Vision-Language-Action (VLA) 模型提供了方法论基础。

5.2 优势

1. **架构简洁:** 最小化修改自标准 GPT 架构, 训练目标接近监督学习
2. **离线友好:** 对离线轨迹数据天然适配, 无需在线交互
3. **多模态可扩展:** 状态、动作、回报各模态使用独立嵌入层, 易于扩展到 VLA 等多模态的架构
4. **信用分配高效:** self-attention 通过 softmax 归一化的 query-key 点积直接建立长程状态-回报关联

5.3 局限与讨论

1. **数据质量依赖:** 若数据中缺少高回报行为, 仅提高 target return 不能保证产生好策略
2. **理论解释性:** 不显式使用 Bellman backup, 并将范式转向了 GPT 的路径, 对于算法可解释性与稳定性没有足够强的理论保证
3. **训练效率:** 在小规模经典控制任务上, Transformer 的训练成本可能高于 DQN、PPO
4. **规模化挑战:** 在长时序、稀疏奖励和复杂机器人环境中, 模型规模 $L \times d_{\text{model}} \times K$ 需与数据规模匹配
5. **任务设定适配性:** 该算法更适合需要有时序记忆的任务, 对于一些经典问题上效率与模型表现不会与经典方法有明显优势甚至表现不如
6. **与 Trajectory Transformer 的对比:** 同期工作 Trajectory Transformer (Janner et al., 2021) 引入状态/回报预测和离散化, 增加了 model-based 组件, 在某些任务上表现更好, 但 DT 的纯 model-free 序列建模范式更为简洁

5.4 本课程复现方案

本课程复现聚焦于在自行设置的 MiniGrid MemoryS17Random 部分可观测记忆任务上复现 DT 核心思想:

1. **数据构造:** 收集不同策略水平的离线轨迹, 计算 return-to-go
2. **模型实现:** GPT 架构 + 三模态嵌入 + causal mask
3. **训练:** 均方误差/交叉熵损失下的动作预测
4. **评估:** Causal Transformer (K=20) vs Behavior Cloning vs Return-Conditioned BC (K=1) vs MLP (K=1)

此方案覆盖了从文献阅读、核心算法复现、实验分析到改进实验的完整流程。

6 参考文献

- [1] Chen, L. et al. **Decision Transformer: Reinforcement Learning via Sequence Modeling.** *NeurIPS*, 2021.
- [2] Vaswani, A. et al. **Attention Is All You Need.** *NeurIPS*, 2017.
- [3] Radford, A. et al. **Improving Language Understanding by Generative Pre-Training.** *OpenAI*, 2018.
- [4] Kumar, A. et al. **Conservative Q-Learning for Offline Reinforcement Learning.** *NeurIPS*, 2020.
- [5] Fu, J. et al. **D4RL: Datasets for Deep Data-Driven Reinforcement Learning.** *arXiv:2004.07219*, 2020.
- [6] Agarwal, R. et al. **An Optimistic Perspective on Offline Reinforcement Learning.** *ICML*, 2020.
- [7] Janner, M. et al. **Offline Reinforcement Learning as One Big Sequence Modeling Problem.** *NeurIPS*, 2021.

- [8] Srivastava, R. K. et al. **Training Agents Using Upside-Down Reinforcement Learning**. *arXiv:1912.02877*, 2019.
- [9] Sutton, R. S. and Barto, A. G. **Reinforcement Learning: An Introduction**. *MIT Press*, 2018.
- [10] Mesnard, T. et al. **Counterfactual Credit Assignment in Model-Free Reinforcement Learning**. *ICML*, 2021.